Assignment-12.1(AI)

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
  - Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
  - Ask AI to include time complexity and space complexity in the function docstring.
  - Verify the generated code with test cases.
- Expected Output:
  - A functional Python script implementing Merge Sort with proper documentation.

lab 9.1task_1.ipy lab 12.1.ipy

lab 12.1.ipy > ...

```python

#Requirements:
#1. Create a function merge_sort(arr) that sorts a list in ascending order.
#2. Use recursion.
#3. Include a docstring explaining time and space complexity.
#4. Print original and sorted list.

def merge_sort(arr):

    if len(arr) > 1:

        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):

            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1

            else:
                arr[k] = right[j]
                j += 1

            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
# Testing

numbers = [38,27,43,3,9,82,10]

print("Original List:", numbers)

merge_sort(numbers)

print("Sorted List:", numbers)
```

```
PS C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/DEL
Python AI/lab 12.1.ipy"
Original List: [38, 27, 43, 3, 9, 82, 10]
Original List: [38, 27, 43, 3, 9, 82, 10]
Sorted List: [3, 9, 10, 27, 38, 43, 82]
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:

- Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
- Include docstrings explaining best, average, and worst-case complexities.
- Test with various inputs.

- Expected Output:
  - Python code implementing binary search with AI-generated comments and docstrings.

lab 12.1.ipy > ...

```python

# TASK 2: BINARY SEARCH

#Write a Python program implementing Binary Search.
#Requirements:
#1. Function binary_search(arr,target)
#2. Return index of element
#3. Return -1 if element not found
#4. Include complexity explanation

def binary_search(arr, target):
    left = 0
    right = len(arr)-1

    while left <= right:

        mid = (left + right)//2

        if arr[mid] == target:
            return mid

        elif arr[mid] < target:
            left = mid + 1

        else:
            right = mid - 1

    return -1
#  Testing

arr = [2,4,6,8,10,12,14]

target = 10

result = binary_search(arr,target)

if result != -1:
    print("Element found at index:",result)

else:
    print("Element not found")

```

```
ROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS    POSTMAN CONSOLE

S C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python
S C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python
lement found at index: 4
```

Task Description #3 (Real-Time Application – Inventory Management System)

- Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:
    1. Quickly search for a product by ID or name.
    2. Sort products by price or quantity for stock analysis.
- Task:
    - Use AI to suggest the most efficient search and sort algorithms for this use case.
    - Implement the recommended algorithms in Python.
    - Justify the choice based on dataset size, update frequency, and performance requirements.
- Expected Output:
    - A table mapping operation → recommended algorithm → justification.
    - Working Python functions for searching and sorting the inventory.

```python
# TASK 3: INVENTORY MANAGEMENT SYSTEM
#Create a Python program for an Inventory Management System.

#Product details:
#product_id
#product_name
#price
#quantity

#Tasks:
#1. Search product by product_id
#2. Sort products by price
#3. Display sorted list
products = [
{"product_id":101,"name":"Laptop","price":50000,"quantity":10},
{"product_id":102,"name":"Mouse","price":500,"quantity":50},
{"product_id":103,"name":"Keyboard","price":1200,"quantity":30},
{"product_id":104,"name":"Monitor","price":15000,"quantity":15}
]


sorted_products = sorted(products,key=lambda x:x["price"])

print("\nProducts Sorted by Price")

for p in sorted_products:
    print(p)


def search_product(pid):

    for p in products:

        if p["product_id"] == pid:
            return p

    return None


print("\nSearch Product")
print(search_product(103))
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS    POSTMAN CONSOLE

SyntaxError: unterminated string literal (detected at line 97)
PS C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python\Pyt

PS C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python\Pyt

Products Sorted by Price
Products Sorted by Price
{'product_id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 50}
{'product_id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 50}
{'product_id': 103, 'name': 'Keyboard', 'price': 1200, 'quantity': 30}
{'product_id': 104, 'name': 'Monitor', 'price': 15000, 'quantity': 15}
{'product_id': 101, 'name': 'Laptop', 'price': 50000, 'quantity': 10}


Search Product
{'product_id': 103, 'name': 'Keyboard', 'price': 1200, 'quantity': 30}
PS C:\Users\DELL\Desktop\Python AI>
```

Task description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:

1. Quickly search patient records using patient ID or name.
2. Sort patients based on severity level or bill amount for prioritization and billing.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms in terms of efficiency and suitability.
- Implement the recommended algorithms in Python.

```python
# TASK 4: SMART HOSPITAL PATIENT MANAGEMENT
#Create a Smart Hospital Patient Management System.

#Patient details:
#patient_id
#name
#severity_level
#bill_amount

#Tasks:
#1. Search patient by patient_id
#2. Sort patients by severity level
patients = [
{"patient_id":1,"name":"Ravi","severity":3,"bill":20000},
{"patient_id":2,"name":"Anita","severity":5,"bill":35000},
{"patient_id":3,"name":"Kiran","severity":2,"bill":15000},
{"patient_id":4,"name":"Sita","severity":4,"bill":25000}
]


sorted_patients = sorted(patients,key=lambda x:x["severity"],reverse=True)

print("\nPatients Sorted by Severity")

for p in sorted_patients:
    print(p)


def search_patient(pid):

    for p in patients:

        if p["patient_id"] == pid:
            return p

    return None


print("\nSearch Patient")
print(search_patient(3))
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS    POSTMAN CONSOLE

PS C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python\Python31
PS C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python\Python31

Patients Sorted by Severity
{'patient_id': 2, 'name': 'Anita', 'severity': 5, 'bill': 35000}
{'patient_id': 4, 'name': 'Sita', 'severity': 4, 'bill': 25000}
{'patient_id': 1, 'name': 'Ravi', 'severity': 3, 'bill': 20000}
{'patient_id': 3, 'name': 'Kiran', 'severity': 2, 'bill': 15000}

Search Patient
{'patient_id': 3, 'name': 'Kiran', 'severity': 2, 'bill': 15000}
PS C:\Users\DELL\Desktop\Python AI>
```

Task Description #5: **University Examination Result Processing System**

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.
2. Sort students based on marks to generate rank lists.

Student Task

- Identify efficient searching and sorting algorithms using AI assistance.
- Justify the choice of algorithms.
- Implement the algorithms in Python.

```python
# TASK 5: UNIVERSITY RESULT PROCESSING
#Create a University Result Processing System.

#Student details:
#roll_number
#name
#marks

#Tasks:
#1. Search student by roll number
#2. Sort students by marks to generate rank list

students = [
{"roll":1,"name":"Rahul","marks":78},
{"roll":2,"name":"Priya","marks":92},
{"roll":3,"name":"Arjun","marks":65},
{"roll":4,"name":"Sneha","marks":88}
]


sorted_students = sorted(students,key=lambda x:x["marks"],reverse=True)

print("\nRank List")

for s in sorted_students:
    print(s)


def search_student(roll):

    for s in students:

        if s["roll"] == roll:
            return s

    return None


print("\nSearch Student")
print(search_student(2))
```

.

```
PS C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Pyt

Rank List
{'roll': 2, 'name': 'Priya', 'marks': 92}

Rank List
{'roll': 2, 'name': 'Priya', 'marks': 92}
{'roll': 4, 'name': 'Sneha', 'marks': 88}
{'roll': 1, 'name': 'Rahul', 'marks': 78}
{'roll': 2, 'name': 'Priya', 'marks': 92}
{'roll': 4, 'name': 'Sneha', 'marks': 88}
{'roll': 1, 'name': 'Rahul', 'marks': 78}
{'roll': 3, 'name': 'Arjun', 'marks': 65}
{'roll': 4, 'name': 'Sneha', 'marks': 88}
{'roll': 1, 'name': 'Rahul', 'marks': 78}
{'roll': 3, 'name': 'Arjun', 'marks': 65}

{'roll': 3, 'name': 'Arjun', 'marks': 65}

Search Student

Search Student
Search Student
{'roll': 2, 'name': 'Priya', 'marks': 92}
```

Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

1. Quickly find an order using order ID.
2. Sort orders based on delivery time or price.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the algorithm selection.
- Implement searching and sorting modules in Python.

lab 12.1.ipy > ...

```python

# TASK 6: ONLINE FOOD DELIVERY SYSTEM
#Create a Python program for an Online Food Delivery Platform.

#Order details:
#order_id
#restaurant_name
#delivery_time
#price

#Tasks:
#1. Search order by order_id
#2. Sort orders by delivery_time
orders = [
{"order_id":1,"restaurant":"Dominos","delivery_time":30,"price":500},
{"order_id":2,"restaurant":"KFC","delivery_time":25,"price":700},
{"order_id":3,"restaurant":"Pizza Hut","delivery_time":40,"price":650},
{"order_id":4,"restaurant":"Burger King","delivery_time":20,"price":450}
]


sorted_orders = sorted(orders,key=lambda x:x["delivery_time"])

print("\nOrders Sorted by Delivery Time")

for o in sorted_orders:
    print(o)


def search_order(order_id):

    for o in orders:

        if o["order_id"] == order_id:
            return o

    return None


print("\nSearch Order")
print(search_order(3))
```

```
PS C:\Users\DELL\Desktop\Python AI> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/DELL/De

Orders Sorted by Delivery Time
{'order_id': 4, 'restaurant': 'Burger King', 'delivery_time': 20, 'price': 450}
{'order_id': 2, 'restaurant': 'KFC', 'delivery_time': 25, 'price': 700}
{'order_id': 1, 'restaurant': 'Dominos', 'delivery_time': 30, 'price': 500}
{'order_id': 3, 'restaurant': 'Pizza Hut', 'delivery_time': 40, 'price': 650}

Search Order
{'order_id': 3, 'restaurant': 'Pizza Hut', 'delivery_time': 40, 'price': 650}
PS C:\Users\DELL\Desktop\Python AI>



{'order_id': 3, 'restaurant': 'Pizza Hut', 'delivery_time': 40, 'price': 650}
PS C:\Users\DELL\Desktop\Python AI>
```